

Python 연동

redis-py

Python에서 Redis를 사용하려면 `redis-py` 라이브러리를 설치합니다.

```
pip install -r requirements.txt
```

항목	내용
라이브러리	<code>redis-py</code>
설치	<code>pip install redis</code>

redis-py는 Redis 공식 Python 클라이언트입니다. Redis 명령어와 거의 같은 이름의 메서드를 제공하므로, Redis CLI에서 배운 명령어를 그대로 활용할 수 있습니다.

연결 설정

```
import redis

r = redis.Redis(host='localhost', port=6379, db=0, decode_responses=True)
print(r.ping()) # True
```

파라미터	기본값	설명
host	'localhost'	Redis 서버 주소
port	6379	Redis 포트
db	0	데이터베이스 번호 (0~15)
decode_responses	False	True 이면 bytes 대신 str 반환

decode_responses=True 를 설정하면 값을 str 로 자동 변환합니다.
기본값 False 이면 bytes 타입으로 반환됩니다.

환경 변수로 연결 설정

코드에 접속 정보를 직접 넣는 대신 `.env` 파일로 분리합니다.

```
REDIS_HOST=localhost  
REDIS_PORT=6379  
REDIS_DB=0  
REDIS_PASSWORD=
```

```
import os
import redis
from dotenv import load_dotenv

load_dotenv()

r = redis.Redis(
    host=os.getenv('REDIS_HOST', 'localhost'),
    port=int(os.getenv('REDIS_PORT', 6379)),
    db=int(os.getenv('REDIS_DB', 0)),
    password=os.getenv('REDIS_PASSWORD') or None,
    decode_responses=True
)
```

String 연동

Redis 명령어와 동일한 이름의 메서드를 사용합니다.

```
# SET / GET
r.set('greeting', 'Hello Redis')
print(r.get('greeting'))          # Hello Redis

# 만료 시간 설정 (ex=초)
r.set('session:abc', 'user:7', ex=3600)
print(r.ttl('session:abc'))      # 남은 초 확인 (~3600)

# 카운터
r.set('views:article:100', 0)
r.incr('views:article:100')
r.incrby('views:article:100', 5)
print(r.get('views:article:100')) # 6
```

List 연동

```
r.delete('queue:orders')

# RPUSH / LRANGE
r.rpush('queue:orders', 'order:1', 'order:2', 'order:3')
print(r.lrange('queue:orders', 0, -1))
# ['order:1', 'order:2', 'order:3']

# LPOP (앞에서 꺼내기)
order = r.lpop('queue:orders')
print(order)                # order:1

# LLEN
print(r.llen('queue:orders'))  # 2
```


Set 연동

```
r.delete('tags:article:100')

# SADD / SMEMBERS
r.sadd('tags:article:100', 'python', 'redis', 'database')
print(r.smembers('tags:article:100'))
# {'python', 'redis', 'database'}

# SISMEMBER
print(r.sismember('tags:article:100', 'python')) # True

# SCARD
print(r.scard('tags:article:100')) # 3

# SREM
r.srem('tags:article:100', 'database')
```

Sorted Set 연동

```
r.delete('ranking:game')

# ZADD: {멤버: 점수} 형태로 전달
r.zadd('ranking:game', {'user:1': 300, 'user:2': 150, 'user:3': 450})

# ZRANGE (오름차순, withscores=True)
print(r.zrange('ranking:game', 0, -1, withscores=True))
# [('user:2', 150.0), ('user:1', 300.0), ('user:3', 450.0)]

# ZREVRANK (내림차순 순위, 0부터 시작)
print(r.zrevrank('ranking:game', 'user:3')) # 0 (1등)

# ZINCRBY
r.zincrby('ranking:game', 100, 'user:2')
print(r.zscore('ranking:game', 'user:2')) # 250.0
```

Hash 연동

```
r.delete('user:7')

# HSET: mapping으로 여러 필드 한번에 저장
r.hset('user:7', mapping={'name': '민수', 'email': 'minsu@example.com', 'level': '1'})

# HGET / HGETALL
print(r.hget('user:7', 'name'))    # 민수
print(r.hgetall('user:7'))
# {'name': '민수', 'email': 'minsu@example.com', 'level': '1'}

# HINCRBY
r.hincrby('user:7', 'level', 1)
print(r.hget('user:7', 'level'))   # 2

# HDEL
r.hdel('user:7', 'email')
```

캐시 패턴 (Cache Aside)

애플리케이션이 Redis를 캐시로 사용하는 가장 일반적인 패턴입니다.

요청 → Redis에서 읽기

└ 캐시 히트(Hit): Redis에서 값 반환

└ 캐시 미스(Miss): DB 조회 → Redis에 저장 → 값 반환

```
import json

def get_product(product_id: int) -> dict:
    cache_key = f'cache:product:{product_id}'
    cached = r.get(cache_key)
    if cached:
        return json.loads(cached)                # 캐시 히트

    product = db_query(product_id)                # 캐시 미스 → DB 조회
    r.set(cache_key, json.dumps(product), ex=300)  # 5분 캐시
    return product
```

Pipeline: 여러 명령 묶어 보내기

```
pipe = r.pipeline()
pipe.set('key1', 'value1')
pipe.set('key2', 'value2')
pipe.get('key1')
pipe.get('key2')
results = pipe.execute()
print(results) # [True, True, 'value1', 'value2']
```

Pipeline을 사용하면 여러 명령을 한 번에 서버로 보내 네트워크 왕복을 줄일 수 있습니다. 대량의 데이터를 처리할 때 성능이 크게 향상됩니다.

순서가 보장되며 중간에 오류가 나도 나머지 명령은 실행됩니다.

원자적 처리가 필요하다면 `r.pipeline(transaction=True)` 를 사용합니다.

ConnectionPool

```
pool = redis.ConnectionPool(  
    host='localhost', port=6379, db=0,  
    decode_responses=True, max_connections=10  
)  
r = redis.Redis(connection_pool=pool)
```

여러 스레드나 요청에서 Redis를 사용할 때 ConnectionPool로 연결을 재사용합니다.

방식	설명
연결 없이 매번 생성	연결 비용 발생, 동시 처리 어려움
ConnectionPool	연결 재사용, 동시 요청 처리 가능

웹 서버처럼 동시 요청이 많은 환경에서는 반드시 풀을 사용합니다.